

- [1. 线性回归 \(Linear Regression\)](#)
 - [1.1 监督学习的基本框架](#)
 - [1.2 高维的线性](#)
 - [1.3 线性回归](#)
 - [1.3.1 损失函数 \(Loss Function\)](#)
 - [1.3.1.1 平方误差损失函数](#)
 - [1.3.1.2 成本函数 \(Cost Function\)](#)
 - [1.3.2 向量化](#)
 - [1.3.3 解决最小化问题 \(Minimization Problem\)](#)
 - [1.3.3.1 代数方法](#)
 - [1.3.3.2 微积分方法](#)
 - [1.4 多项式特征映射 \(Polynomial Feature Mapping\)](#)
 - [1.4.1 M=0的多项式特征映射](#)
 - [1.4.2 M=1的多项式特征映射](#)
 - [1.4.3 M=3的多项式特征映射](#)
 - [1.4.4 M=9的多项式特征映射](#)
 - [1.4.5 总结](#)
 - [1.5 正则化 \(Regularization\)](#)
 - [1.5.1 正则化器 \(Regularizer\)](#)
 - [1.5.2 岭回归 \(Ridge Regression\)](#)
 - [1.5.3 总结](#)
 - [1.5.3.1 梯度下降](#)
 - [1.5.3.1.1 学习率 \$\alpha\$](#)
 - [1.5.3.1.2 训练曲线 \(Training Curves\)](#)
 - [1.5.3.1.3 随机梯度下降 \(Stochastic Gradient Descent, SGD\)](#)
 - [1.5.3.1.3.1 学习率对SGD的影响](#)
 - [1.5.3.1.4 小批量 \(mini-batch\) 梯度下降](#)

1. 线性回归 (Linear Regression)

线性回归输入的是线性函数，所以线性回归模型假设输入特征和输出目标之间存在线性关系。

因此其在预测衡量值目标时用于预测一个连续的数值。

与 k-最近邻 (k-NN) 作为一个完整的算法不同，线性回归展示了一种模块化方法，这种方法将在课程中不断使用。步骤如下：

1. 选择一个描述感兴趣变量之间关系的模型。在线性回归中，这个模型是一个线性函数。
2. 定义一个损失函数，量化模型拟合数据的好坏。损失函数衡量模型预测值与实际值之间的差异。
3. 选择一个正则化器，表示我们对不同候选模型（或数据解释）的偏好。正则化有助于防止过拟合，通过在损失函数中添加一个额外的惩罚项来实现。
4. 拟合一个最小化损失函数并满足正则化器约束/惩罚的模型，可能使用优化算法。这一步涉及找到模型参数，使得损失函数达到最小值。

通过混合和匹配这些模块化组件，我们可以创建许多新的机器学习方法。这种方法的灵活性允许我们根据不同的问题和数据集调整模型、损失函数和正则化器，从而开发出适合特定任务的算法。

1.1 监督学习的基本框架

在监督学习中：输入 $x \in X$ ，通常是特征向量（或协变量）。

目标 $t \in T$ ，也称为响应（response）、结果（outcome）、输出（output）或类别（class）。

监督学习的目标是学习一个函数

$f: X \rightarrow T$ ，使得 $t \approx y = f(x)$ ，基于一些数据 $D = \{(x_{(i)}, t_{(i)}), i = 1, 2, \dots, N\}$ 。

在线性回归中，我们使用特征 $x = (x_1, \dots, x_D) \in R^D$ 的线性函数来预测目标值 $t \in R$ 的预测值 y ，公式如下： $y = f(\mathbf{x}) = \sum_j w_j x_j + b$ 。

y ：预测值（prediction）。

w ：权重（weights），表示每个特征 x_j 对预测结果的影响程度。

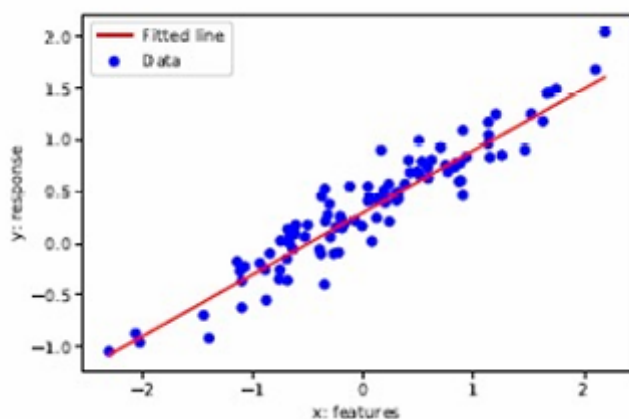
b ：偏置项（bias 或 intercept），表示当所有特征值为零时的预测值。

这里 w 和 b 一起构成了模型的参数（parameters）。

我们希望预测值 y 接近目标值 t ，即 $y \approx t$ 。

1.2 高维的线性

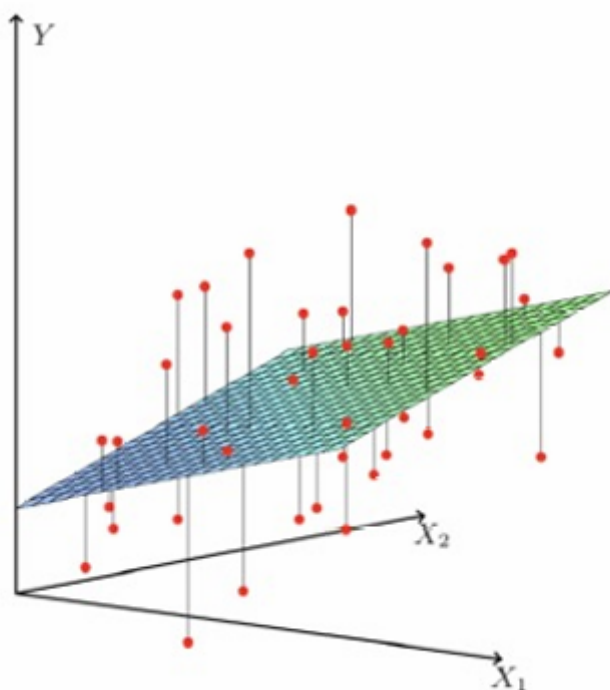
当只有一个特征的时候，我们很容易理解，这里是简单的线性关系。



这里 $y = wx + b$ ，表示 y 是 x 的线性函数，其中 w 是权重， x 和 b 是实数。

这里 y 与 x 的线性关系很容易理解。

那如果维度变高呢，下图展示了 D 个特征。



这里 $y = w^T x + b$ 表示 y 是特征向量 x 的线性函数，其中 w 是权重向量， x 是特征向，需要注意的是这里向量 w 的上标 T 表示转置（Transpose），即将行向量转换为列向量，或反之，而不是次方。

这里 y 与 x 的线性关系依然是存在的，这里线性关系是通过权重向量 w 和特征向量 x 的点积来表示的。

1.3 线性回归

我们现在有一个数据集 D ：包含 N 个样本，每个样本由输入 $x^{(i)}$ 和目标 $t^{(i)}$ 组成。

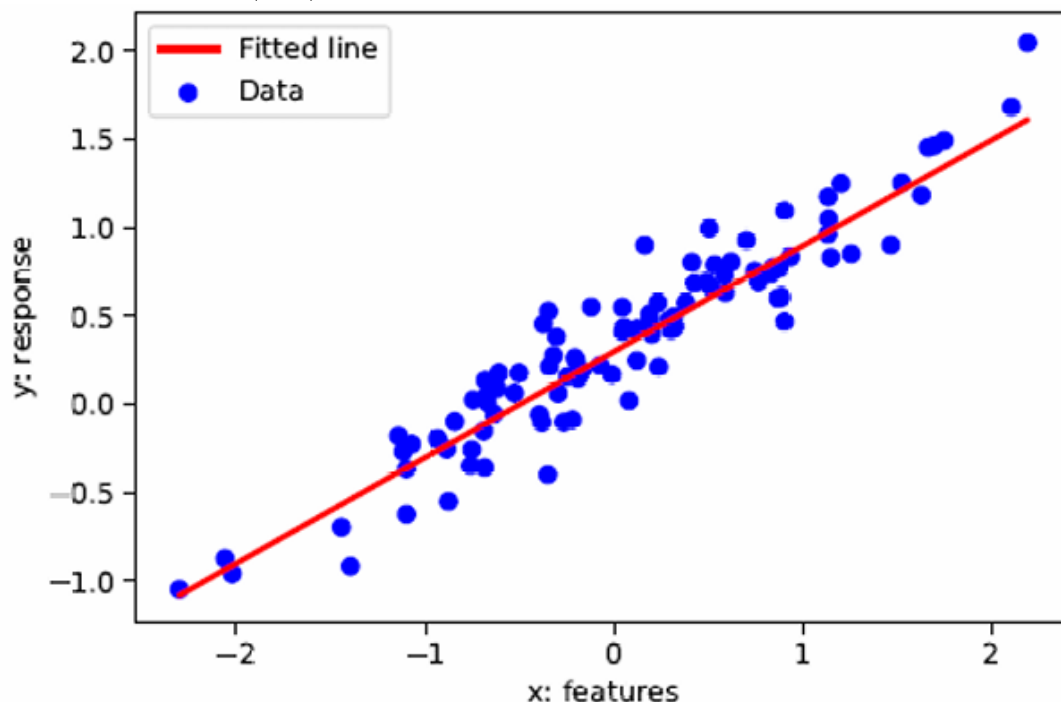
这里输入 $x^{(i)} = (x_1^{(i)}, x_2^{(i)}, \dots, x_D^{(i)})^T \in R^D$ ：是一个 D 维向量，表示特征（例如年龄、身高）。

目标 $t^{(i)} \in R$ ：是一个实数，表示响应变量（例如收入）。

我们使用 $x^{(i)}$ 的线性函数来预测 $t^{(i)}$ ： $t^{(i)} \approx y^{(i)} = w^T x^{(i)} + b$

不同的 (w, b) 定义了不同的直线。

我们希望找到“最佳”的直线 (w, b) 。



1.3.1 损失函数（Loss Function）

为了找到“最佳”的直线 (w, b) ，我们可以使用损失函数 $L(y, t)$ 。损失函数 $L(y, t)$ 定义了对于某个样本 x ，算法预测的 y 与实际目标 t 之间的差异有多大。

1.3.1.1 平方误差损失函数

公式： $L(y, t) = 1/2(y - t)^2$

这里， $y - t$ 是残差（residual），表示预测值与实际值之间的差异。我们希望这个差异尽可能小。

$1/2$ 因子只是为了使计算更方便，不会影响优化过程。

1.3.1.2 成本函数（Cost Function）

成本函数是所有训练样本上的损失函数的平均值。

公式： $J(\mathbf{w}, b) = \frac{1}{2N} \sum_{i=1}^N (y^{(i)} - t^{(i)})^2$,

另一种表示方式： $J(\mathbf{w}, b) = \frac{1}{2N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}^{(i)} + b - t^{(i)})^2$

成本函数也可以因此称之为“成本”（cost）、经验损失（empirical loss）或平均损失（average loss）。

1.3.2 向量化

当我们展开损失函数的公式我们就会得到一个非常复杂的式子：

$$\frac{1}{2N} \sum_{i=1}^N \sum_{j=1}^D \left(w_j x_j^{(i)} + b - t^{(i)} \right)^2.$$

在计算时如果使用Python的循环进行计算就会较慢，为了提高效率，可以通过向量和矩阵表达算法来向量化算法。

首先将权重 w 和输入 x 表示为向量： $w = (w_1, \dots, w_D)^T, x = (x_1, \dots, x_D)^T$ ，因此通过向量和矩阵表达算法简化预测公式 $y = w^T x + b$ ，并且使用Numpy的点积函数可以得到 $y = np.dot(w, x) + b$ 。

通过向量化，我们可以是我们的代码和方程更简单、易读（消除了对临时变量和索引的需求），而且这样的代码执行速度更快（更少的解释器开销、使用了高度优化的线性代数库、矩阵算法在GPU上运行非常快）。

当我们的数学推导涉及逐元素（element-wise）操作或者需要提高性能时就可以使用向量化。

逐元素操作指的是对向量或矩阵中的每个元素分别进行相同的运算。例如，如果你有一个向量

$a = (a_1, a_2, \dots, a_D)$ 和一个标量 b ，那么逐元素的加法就是创建一个新的向量

$a = (a_1 + b, a_2 + b, \dots, a_D + b)$ ，其中每个元素 c_i 是 a_i 和 b 的和。

进行向量化时，我们将所有训练样本组织成一个设计矩阵 X ，其中每一行代表一个训练样本，每一列代表一个特征。

将所有目标值组织成一个目标向量 t 。

示例如下：

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}^{(1)\top} \\ \mathbf{x}^{(2)\top} \\ \mathbf{x}^{(3)\top} \end{pmatrix} = \begin{pmatrix} 8 & 0 & 3 & 0 \\ 6 & -1 & 5 & 3 \\ 2 & 5 & -2 & 8 \end{pmatrix}$$

每一列代表一个特征，每一行代表一个训练样本（向量）。

$$\mathbf{X}\mathbf{w} + b\mathbf{1} = \begin{pmatrix} \mathbf{w}^\top \mathbf{x}^{(1)} + b \\ \vdots \\ \mathbf{w}^\top \mathbf{x}^{(N)} + b \end{pmatrix} = \begin{pmatrix} y^{(1)} \\ \vdots \\ y^{(N)} \end{pmatrix} = \mathbf{y}$$

这里， w 是权重向量， b 是偏置项， $\mathbf{1}$ 是一个全1的向量，用于添加偏置项。

通过矩阵乘法 Xw 和向量加法 $b\mathbf{1}$ ，可以一次性计算整个数据集的预测值 y 。

计算整个数据集上的平方误差成本：

$$\mathbf{y} = \mathbf{X}\mathbf{w} + b\mathbf{1}$$

$$J = \frac{1}{2N} \|\mathbf{y} - \mathbf{t}\|^2$$

这里， y 是预测值， X 是设计矩阵， w 是权重向量， b 是偏置项， $\mathbf{1}$ 是一个全1的列向量， t 是目标向量。

损失函数 J 计算了预测值 y 与实际目标值 t 之间的平方误差的平均值。

有时我们可能使用不带归一化因子的损失函数 $J = \frac{1}{2} \|\mathbf{y} - \mathbf{t}\|^2$ ，这对应于损失的总和，而不是平均损失。最小化器不依赖于样本数量 N （但优化过程可能会）。

我们可以在设计矩阵中添加一列1，将偏置项和权重合并，方便地写为：

$$\mathbf{X} = \begin{bmatrix} 1 & \mathbf{x}^{(1)\top} \\ 1 & \mathbf{x}^{(2)\top} \\ \vdots & \vdots \\ 1 & \mathbf{x}^{(N)\top} \end{bmatrix} \in \mathbb{R}^{N \times (D+1)}$$

$$\mathbf{w} = \begin{bmatrix} b \\ w_1 \\ w_2 \\ \vdots \end{bmatrix} \in \mathbb{R}^{D+1}$$

这样，我们的预测简化为 $y = Xw$ 。

1.3.3 解决最小化问题 (Minimization Problem)

我们现在如何找到使成本函数 (Cost Function) 达到最小值的参数呢？

我们有两种常见的数学方法：代数方法 (Algebraic) 和微积分方法 (Calculus)。

1. 代数方法 (Algebraic)：

使用不等式来证明最小值。例如，要证明某个值 θ^* 最小化了函数 $J(\theta)$ ，需要展示对于所有的 θ ，都有 $J(\theta) \geq J(\theta^*)$ 。

2. 微积分方法 (Calculus)：

对于光滑函数 (如果存在最小值)，其最小值出现在临界点，即导数为零的点。

解决方案可能是直接的或迭代的：

有时我们可以直接找到最优参数 (例如，将梯度设为零并封闭形式求解)。我们称这种解为直接解 (Direct Solution)。

我们也可能使用迭代优化技术，逐步接近解。稍后我们会回到这个主题。

1.3.3.1 代数方法

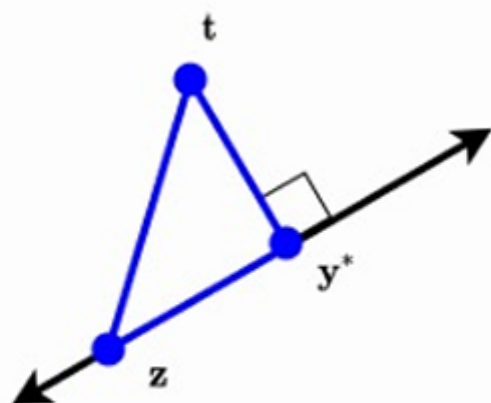
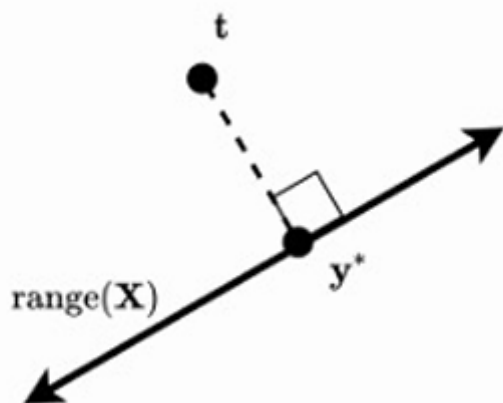
我们现场使用线性代数方法。

我们寻找权重向量 w 来最小化 $\|Xw - t\|^2$ ，这等价于最小化 $\|Xw - t\|$ 。

$\text{range}(X) = \{Xw | w \in \mathbb{R}^D\}$ ，这是 Xw 的所有可能值，其中 $\mathbb{R}^D$ 是 $\mathbb{R}^N$ 的一个 D 维子空间。

我们通过正交投影找到子空间 $\text{range}(X)$ 中任意点 $t \in \mathbb{R}^N$ 的最近点 $y^* = Xw^*$ 。

下图展示了图中展示了点 t 到子空间 $\text{range}(X)$ 的正交投影 y^* 。



$$(\mathbf{y}^\top - \mathbf{t})\mathbf{X}\mathbf{w}, \quad \forall \mathbf{w} \in \mathbb{R}^D.$$

这里给出证明以确定 y^* 是 t 的最近点

首先 $z = Xw$, 然后根据毕达哥拉斯定理 (Pythagorean theorem) 和平凡不等式($x^2 \geq 0$):

$$\|\mathbf{z} - \mathbf{t}\|^2 = \|\mathbf{y}^\top - \mathbf{t}\|^2 + \|\mathbf{y}^\top - \mathbf{z}\|^2 \geq \|\mathbf{y}^\top - \mathbf{t}\|^2$$

然后我们回到前面 $(\mathbf{y}^\top - \mathbf{t})\mathbf{X}\mathbf{w}, \quad \forall \mathbf{w} \in \mathbb{R}^D$, 所以矩阵 X 的列向量与 $(y^* - t)$ 正交。

因此我们可以得到: 1. $\mathbf{X}^\top(\mathbf{y}^\top - \mathbf{t}) = 0$

$$2. \mathbf{X}^\top \mathbf{X} \mathbf{w}^\top - \mathbf{X}^\top \mathbf{t} = 0$$

$$3. \mathbf{X}^\top \mathbf{X} \mathbf{w}^\top = \mathbf{X}^\top \mathbf{t}$$

4. $\mathbf{w}^\top = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{t}$, 因此最优权重 w^* 可以通过求解正规方程得到。

虽然这个解法很干净且推导容易记忆, 但有些临时拼凑。

另一方面, 微积分方法的工具可以广泛应用于可微分的损失函数。

1.3.3.2 微积分方法

我们先复习一下微积分知识。

偏导数 (Partial Derivative) 是关于其一个参数的多变量函数的导数。它衡量当其中一个变量变化而其他变量保持不变时, 函数的变化率。

$$\frac{\partial}{\partial x_1} f(x_1, x_2) = \lim_{h \rightarrow 0} \frac{f(x_1+h, x_2) - f(x_1, x_2)}{h}.$$

所以我们在计算偏导数时, 将其他变量视为常数, 只对一个变量求导数。

因此对于预测 y 的偏导数我们对权重 w_j 进行偏导数: $\frac{\partial y}{\partial w_j} = \frac{\partial}{\partial w_j} \left[\sum_{j'} w_{j'} x_{j'} + b \right] = x_j$, 这表示预测值 y 关于权重 w_j 的偏导数等于输入特征 x_j 。

对偏置 b 进行偏导数: $\frac{\partial y}{\partial b} = \frac{\partial}{\partial b} \left[\sum_{j'} w_{j'} x_{j'} + b \right] = 1$, 这表示预测值 y 关于偏置 b 的偏导数等于1。

我们再复习一下链式法则（Chain Rule），其允许我们将一个复杂函数的导数分解为更简单函数的导数的乘积。

$$\frac{\partial \mathcal{L}}{\partial w_j} = \frac{\partial \mathcal{L}}{\partial y} \frac{\partial y}{\partial w_j}$$

这里，第一项是损失函数 $\mathcal{L}$ 关于预测值 y 的导数，后一项是预测值 y 关于 w_j 的导数。

通过链式法则，我们可以得到：

$$\frac{\partial \mathcal{L}}{\partial w_j} = (y - t)x_j$$

类似地，对于偏置 b 的导数：

$$\frac{\partial \mathcal{L}}{\partial b} = y - t$$

对于成本函数的导数，利用线性和对数数据点的平均值：

$$\frac{\partial J}{\partial w_j} = \frac{1}{N} \sum_{i=1}^N (y^{(i)} - t^{(i)})x_j^{(i)}$$

$$\frac{\partial J}{\partial b} = \frac{1}{N} \sum_{i=1}^N (y^{(i)} - t^{(i)})$$

这里， J 是成本函数， N 是数据点的数量， $y^{(i)}$ 是第 i 个样本的预测值， $t^{(i)}$ 是第 i 个样本的实际目标值。

最小值必须出现在偏导数为零的点：

$$\frac{\partial J}{\partial w_j} = 0 \quad (\forall j), \quad \frac{\partial J}{\partial b} = 0.$$

如果 $\frac{\partial J}{\partial w_j} = 0$ ，可以通过改变 w_j 来减少成本。

梯度是函数的偏导数向量，指向函数增长最快的方向。

对于函数 $f: \mathbb{R}^D \rightarrow \mathbb{R}$ ，梯度 $\nabla f(\mathbf{w})$ 表示为： $\left(\frac{\partial}{\partial w_1} f(\mathbf{w}), \dots, \frac{\partial}{\partial w_D} f(\mathbf{w}) \right)^\top$

这里 $\frac{\partial}{\partial w_j} f(\mathbf{w})$ 是函数 f 关于权重 w_j 的偏导数。

下面介绍海森矩阵（Hessian Matrix），海森矩阵是函数的二阶偏导数矩阵，类似于多元函数的二阶导数。

$\nabla^2 f(\mathbf{w}) \in \mathbb{R}^{D \times D}$ 是一个矩阵，其中 $[\nabla^2 f(\mathbf{w})]_{ij}$ 表示为： $[\nabla^2 f(\mathbf{w})]_{ij} = \frac{\partial^2}{\partial w_i \partial w_j} f(\mathbf{w})$

这里 $\frac{\partial^2}{\partial w_i \partial w_j} f(\mathbf{w})$ 是函数 f 关于权重 w_i 和 w_j 的二阶偏导数。

我们寻找权重向量 w 来最小化损失函数 $J(\mathbf{w}) = \frac{1}{2} \|\mathbf{X}\mathbf{w} - \mathbf{t}\|^2$

这里， X 是设计矩阵， w 是权重向量， t 是目标向量。

对 w 求梯度（梯度是损失函数关于参数的偏导数向量），我们得到：

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \mathbf{X}^\top \mathbf{X} \mathbf{w} - \mathbf{X}^\top \mathbf{t} = 0$$

这个方程表示损失函数关于权重 w 的梯度为零时的点，即损失函数的极小值点。

通过求解上述方程，我们可以得到最优权重：

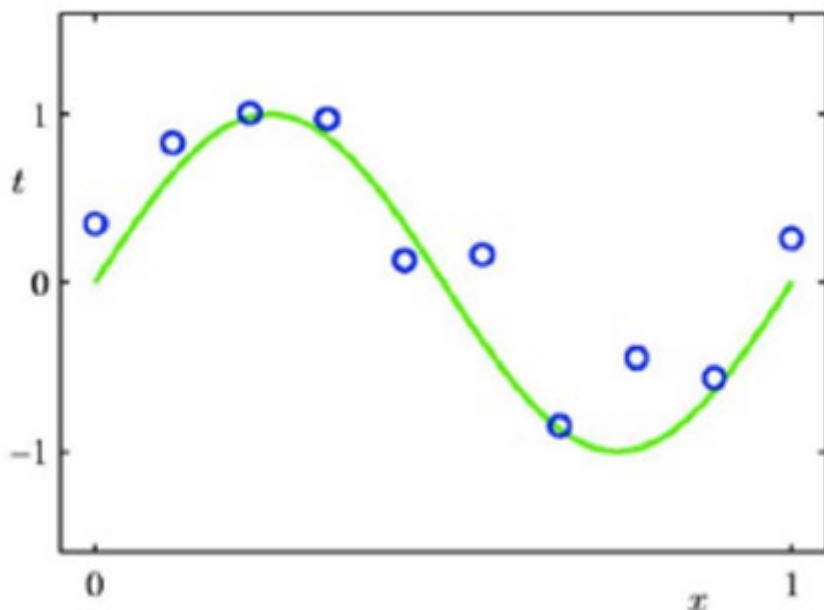
$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{t}$$

线性回归是少数几个允许直接解的模型之一。

通过微积分方法，特别是通过计算梯度并求解正规方程，我们可以直接找到线性回归问题的解。

1.4 多项式特征映射（Polynomial Feature Mapping）

输入和输出之间的关系可能不是线性的，这意味着简单的线性模型可能无法很好地拟合数据。



我们可以将输入特征映射到另一个空间，这便是特征映射（或基拓展） $\varphi(\mathbf{x}) : \mathbb{R}^D \rightarrow \mathbb{R}^d$ 。

在这个新空间中，我们可以将映射后的特征作为线性回归过程的输入。

当 $x \in \mathbb{R}$ 时，我们可以使用多项式特征映射来处理非线性关系。

例如我们有一个预测函数 $y = w_0 + w_1x + w_2x^2 + \cdots + w_Mx^M = \sum_{i=0}^M w_i x^i$

特征映射 $\varphi(\mathbf{x})$ 定义为：

$$\varphi(\mathbf{x}) = [1, \mathbf{x}, \mathbf{x}^2, \cdots, \mathbf{x}^M]^\top$$

这里 $\varphi(\mathbf{x})$ 是一个向量，包含从常数项到 x 的 M 次幂的项。

尽管特征映射后的模型是非线性的，我们仍然可以使用线性回归来拟合权重 $\mathbf{w}$ ，因为 $y = \varphi(\mathbf{x})^\top \mathbf{w}$ 是关于 $\mathbf{w}$ 的线性函数。

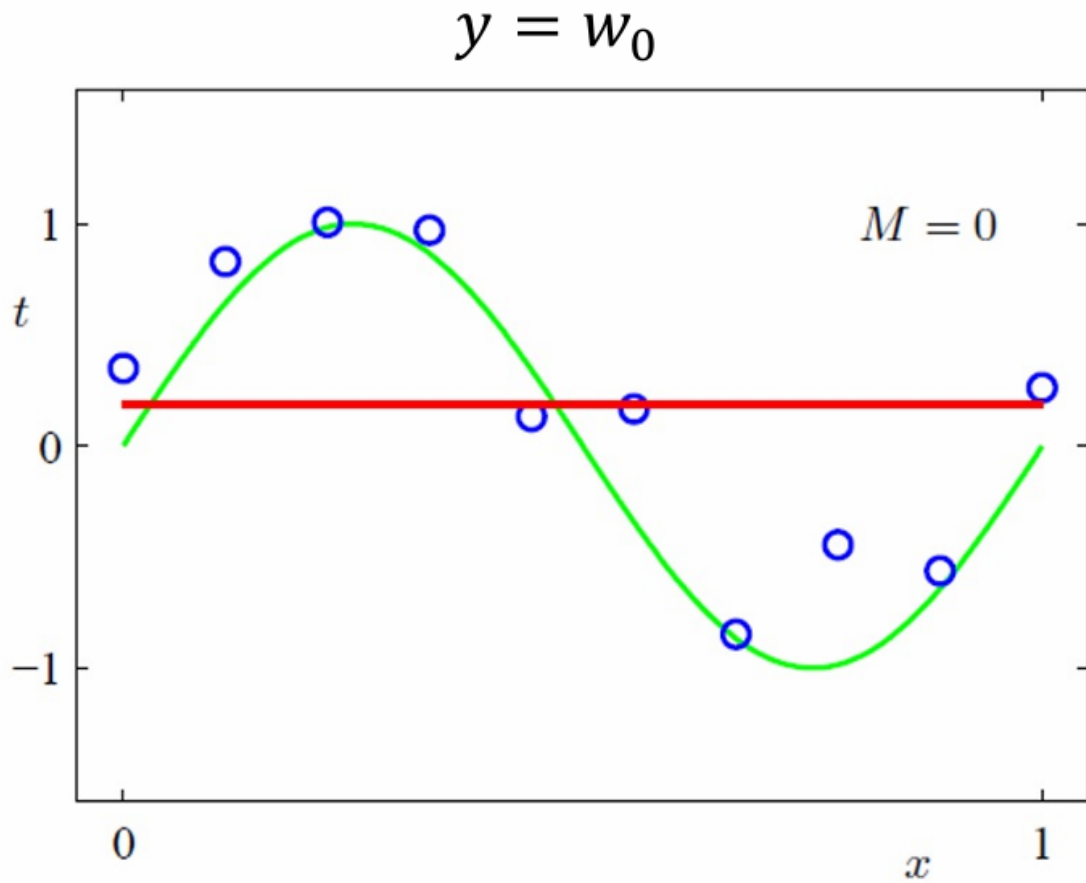
在一般情况下， φ 可以是任何函数。这里给出一个例子：

$$\varphi(\mathbf{x}) = [1, \sin(2\pi\mathbf{x}), \cos(2\pi\mathbf{x}), \sin(4\pi\mathbf{x}), \cdots]^\top$$

1.4.1 M=0的多项式特征映射

当 $M = 0$ 时，特征映射 $\varphi(x)$ 只包含一个常数项，即： $\varphi(x) = [1]$ 这意味着模型只使用一个特征，即常数项1，而不使用输入 x 的任何幂。

由于只使用常数项特征，模型实际上是线性的，即 $y = w$ ，如下图红线所示。



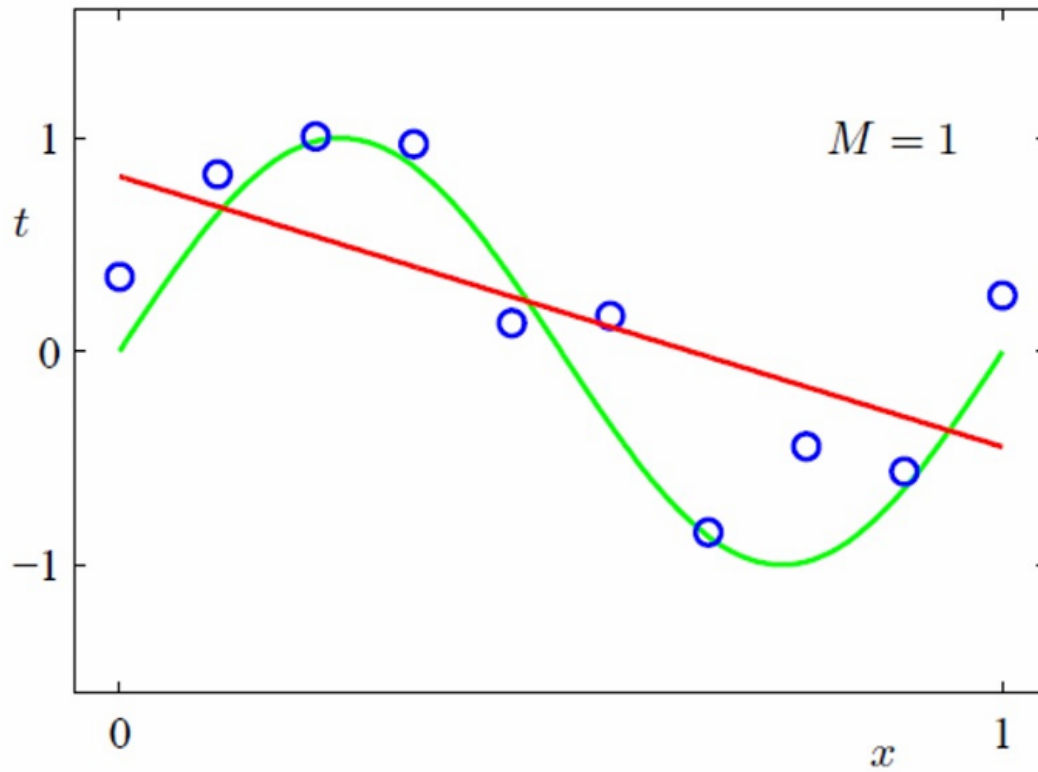
其无法很好地拟合非线性的数据分布。

1.4.2 $M=1$ 的多项式特征映射

当 $M = 1$ 时，特征映射 $\varphi(x)$ 包含一个常数项和一个一次项，即： $\varphi(x) = [1, x]$ 。

下图红线展示了这个线性模型，拟合情况有所进步，但仍有误差。

$$y = w_0 + w_1 x$$



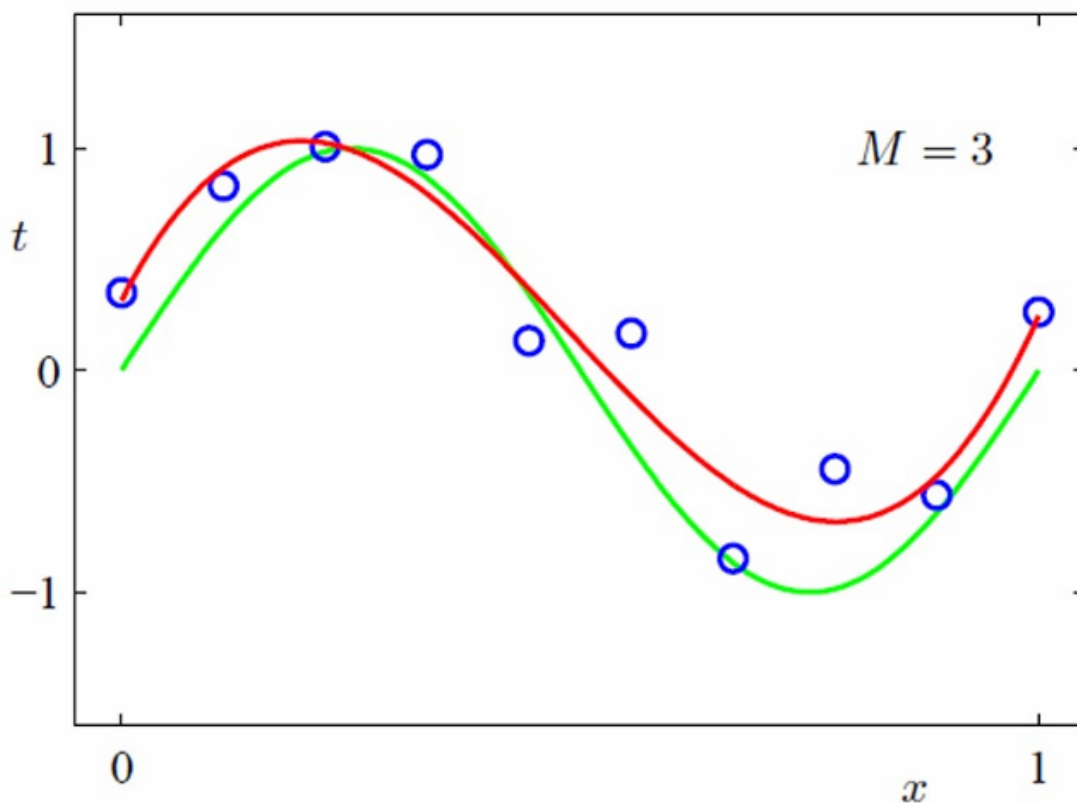
1.4.3 M=3的多项式特征映射

当 $M = 3$ 时，特征映射 $\varphi(x)$ 包含一个常数项、一次项、二次项和三次项，即：

$$\varphi(x) = [1, x, x^2, x^3]。$$

当 $M = 3$ 时，模型可以更好拟合数据了。

$$y = w_0 + w_1x + w_2x^2 + w_3x^3$$



1.4.4 M=9的多项式特征映射

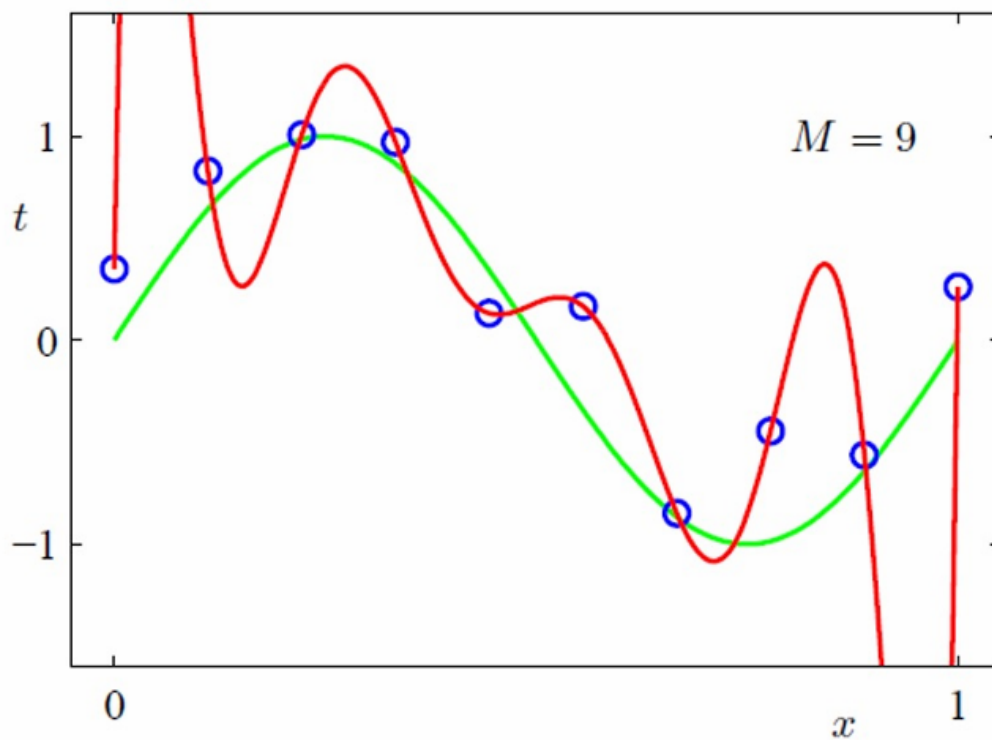
当 $M = 9$ 时，特征映射 $\varphi(x)$ 包含一个常数项和从一次项到九次项，即：

$$\varphi(x) = [1, x, x^2, x^3, \dots, x^9]。$$

通过增加多项式的次数，模型能够捕捉到数据中的更复杂的非线性关系，从而提高预测的准确性。然而，需要注意的是，过高的多项式次数可能会导致过拟合（在中间拟合效果好，两端拟合情况差），即

模型在训练数据上表现很好，但在新数据上的泛化能力差。

$$y = w_0 + w_1x + w_2x^2 + w_3x^3 + \cdots + w_9x^9$$

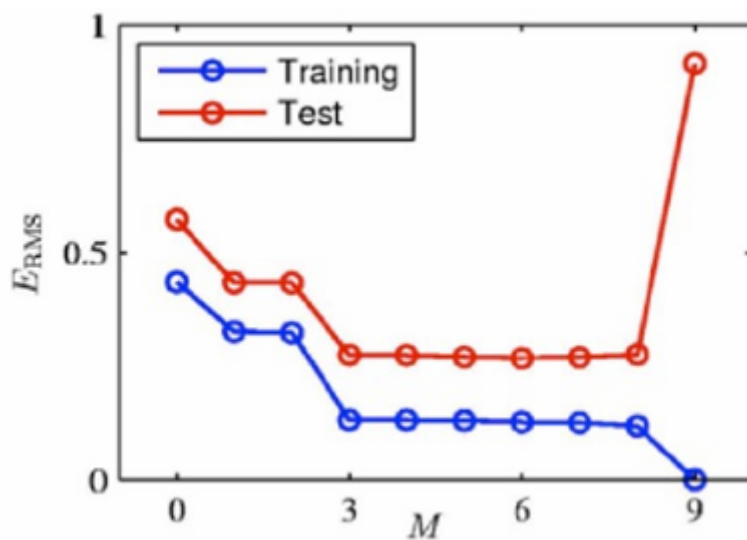


1.4.5 总结

当 $M = 0$ 时，模型过于简单，无法很好地拟合数据，这便是欠拟合（Underfitting）

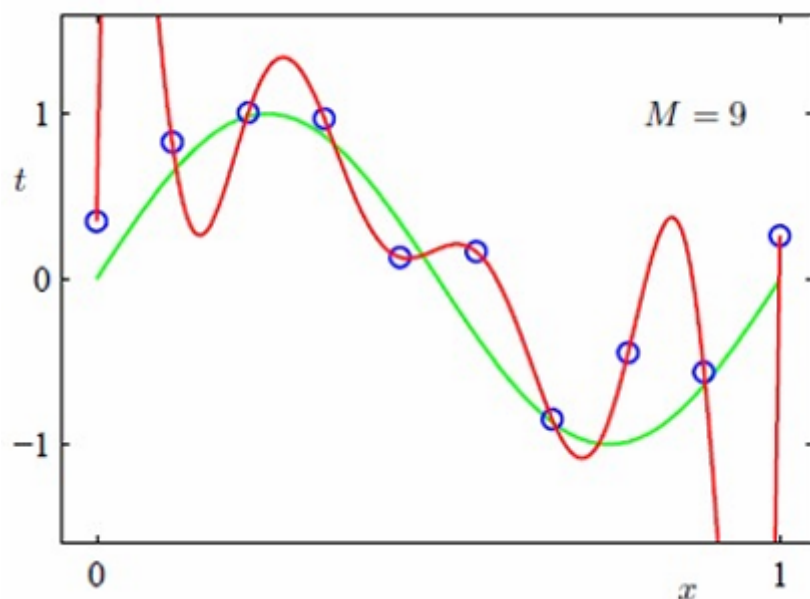
当 $M = 9$ 时，模型过于复杂，能够完美拟合训练数据，但可能无法很好地泛化到新数据，这便是过拟合（Overfitting）。

当 $M = 3$ 时，模型能够很好地拟合数据，并且在测试集上也表现良好，说明模型泛化能力较好。



当 $M = 0$ 时，模型过于简单，权重较小。

随着 M 增加，权重的绝对值显著增大，表明模型变得更加复杂，在数据点之间，函数会表现出大的振荡（oscillations）。



1.5 正则化 (Regularization)

首先我们可以控制模型的复杂度来防止过拟合：

多项式的次数 M ：控制模型复杂度的关键因素。多项式的次数越高，模型的复杂度越高，模型能够捕捉到数据中的更多细节和模式。

超参数 M ：类似于k-最近邻算法中的 k ，是一个需要调整的超参数。可以通过验证集来调整 M 的值，以找到最佳的模型复杂度。

我们也可以限制参数数量来防止过拟合：

限制参数数量或基函数数量（ M ：是一种控制模型复杂度的简单方法。通过限制模型中参数的数量或基函数的数量，可以避免模型过于复杂，从而减少过拟合的风险。

为了防止机器学习模型过拟合，我们也可以使用正则化从而提高模型的泛化能力。

这种方式允许模型保持较高的复杂度，但通过引入正则化项来控制模型的复杂度。

1.5.1 正则化器 (Regularizer)

我们使用正则化器 (Regularizer)：一种量化我们对一个假设相对于另一个假设的偏好程度的函数。正则化器通常是一个惩罚项，添加到损失函数中，以惩罚模型的复杂度。

通过选择L2惩罚作为正则化项，我们可以鼓励模型的权重（参数）保持较小的值。这有助于防止模型对训练数据过度拟合。

L2惩罚的公式是 $\mathcal{R}(\mathbf{w}) = \frac{1}{2} \|\mathbf{w}\|_2^2 = \frac{1}{2} \sum_j w_j^2$ 表示权重向量 w 的平方欧几里得范数（即权重的平方和）。

这里的 $\frac{1}{2} \|\mathbf{w}\|_2^2$

正则化成本函数在拟合数据和权重范数之间做出权衡。公式为

$$\mathcal{J}_{\text{reg}}(\mathbf{w}) = \mathcal{J}(\mathbf{w}) + \lambda \mathcal{R}(\mathbf{w}) = \mathcal{J}(\mathbf{w}) + \frac{\lambda}{2} \sum_j w_j^2$$

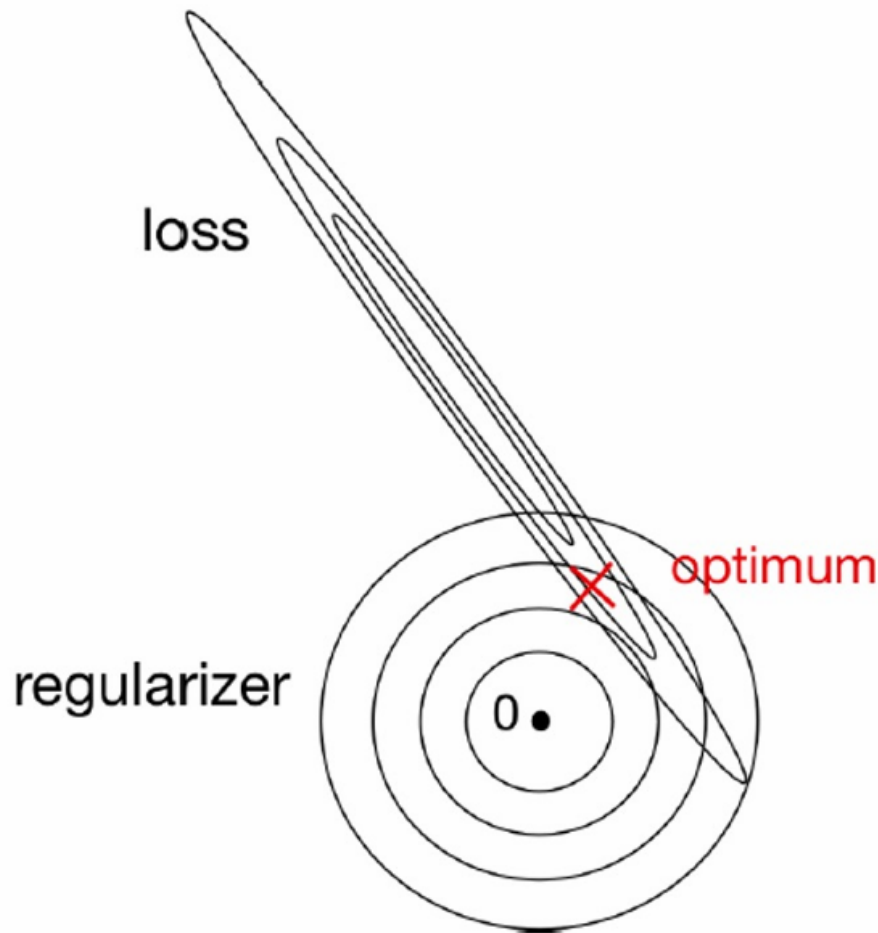
其中， $\mathcal{J}(\mathbf{w})$ 是原始的成本函数， λ 是正则化参数，用于控制正则化项的强度。

这里 λ 是个超参数，可以通过验证集进行调整以找到最佳值。

如果模型对训练数据的拟合不好， $\mathcal{J}$ 会很大。如果最优权重值很高， R 也会很大。

较大的 λ 值会更多地惩罚权重值。

下图展示了一个L2正则化的几何图。



图中loss的椭圆形区域表示损失函数的等高线图，其中损失值随着距离中心点的增加而增加。
regularizer表示正则化项所施加的约束。正则化项试图将模型参数推向原点（0点），因为正则化项随着参数值的增大而增大。
在没有正则化的情况下，最优解可能会位于损失函数等高线的边缘，这可能导致过拟合。
通过引入L2正则化，最优解会被拉向原点，从而使得模型参数值更小，有助于防止过拟合。

1.5.2 岭回归 (Ridge Regression)

通过L2正则化可以改进普通最小二乘法，最小二乘法的损失函数定义为 $\mathcal{J}(\mathbf{w}) = \frac{1}{2N} \|\mathbf{X}\mathbf{w} - \mathbf{t}\|^2$ ，其中 $\mathbf{X}$ 是特征矩阵， $\mathbf{w}$ 是权重向量， $\mathbf{t}$ 是目标向量， N 是样本数量。

当 $\lambda > 0$ （即引入正则化）时，正则化成本函数为：

$$\mathbf{w}_{\lambda}^{\text{Ridge}} = \arg \min_{\mathbf{w}} \mathcal{J}_{\text{reg}}(\mathbf{w}) = \arg \min_{\mathbf{w}} \frac{1}{2N} \|\mathbf{X}\mathbf{w} - \mathbf{t}\|^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2$$

这里增加了一个正则化项 $\frac{\lambda}{2} \|\mathbf{w}\|_2^2$ 有助于防止模型过拟合，因为它惩罚了较大的权重值，从而鼓励模型学习较小的权重。

$\mathbf{w}_{\lambda}^{\text{Ridge}} = (\mathbf{X}^{\top} \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^{\top} \mathbf{t}$ ，其中 $\mathbf{I}$ 是单位矩阵， λ 是正则化参数。

当 $\lambda = 0$ 时， $\mathcal{J}_{\text{reg}}(\mathbf{w})$ 简化为最小二乘问题的解。

另一种常见的公式形式是：

$$\arg \min_{\mathbf{w}} \frac{1}{2} \|\mathbf{X}\mathbf{w} - \mathbf{t}\|^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2$$

这种情况下的解仍然是：

$$\mathbf{w}_{\lambda}^{\text{Ridge}} = (\mathbf{X}^{\top} \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^{\top} \mathbf{t}.$$

1.5.3 总结

关于线性回归现在我们总结解决方案的步骤如下：

1. 选择模型和损失函数：

在机器学习中，首先要选择一个模型（例如线性回归模型）和一个损失函数（例如平方误差损失），这两者定义了模型如何拟合数据。

2. 构建优化问题：

根据模型和损失函数，构建一个优化问题，目的是找到模型参数（如权重和偏置），使得损失函数最小化。

3. 解决最小化问题的两种策略：

直接解法：通过将损失函数的导数（梯度）设为零来找到最小值。对于某些问题，可以直接求解得到解析解。

梯度下降：对于更复杂的问题，可能需要使用梯度下降等迭代优化算法来找到损失函数的最小值。

（梯度下降是下一个要讨论的主题）

4. 向量化算法：

将算法用线性代数的形式表示，这样可以利用矩阵运算来简化计算，提高效率。

5. 使用特征增强线性模型：

通过添加或转换特征（例如多项式特征、交互特征等），可以增强线性模型的表达能力，使其能够拟合更复杂的数据关系。

6. 通过添加正则化项提高泛化能力：

为了防止模型过拟合，可以通过添加正则化项（如L1或L2正则化）来限制模型的复杂度，从而提高模型在未见过的数据上的泛化能力。

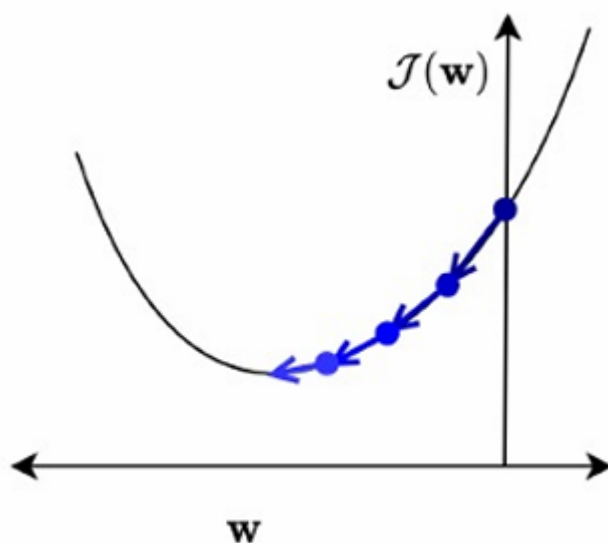
1.5.3.1 梯度下降

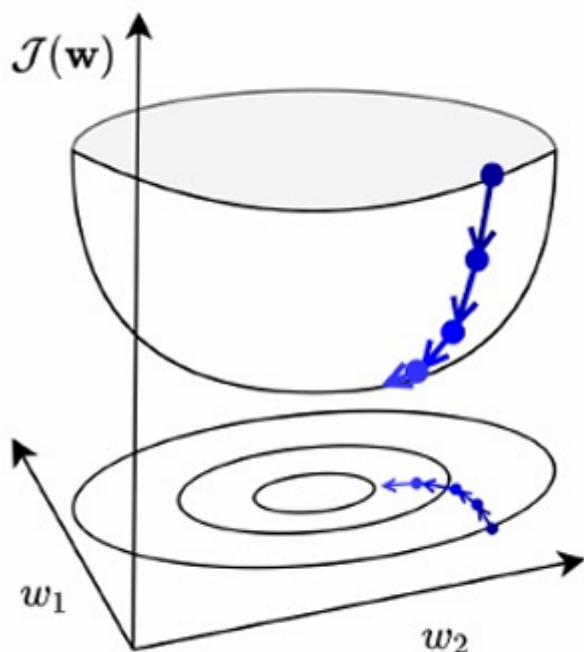
梯度下降是一种迭代算法，用于最小化成本函数。它比直接求解（如将导数设为零）更广泛适用，因为很多情况下直接求解没有明确的解析解。

因此我们会反复应用更新规则，直到满足某个停止条件（如达到最大迭代次数或梯度变化很小）。

我们通常会将权重初始化为一个合理的值（例如，所有权重初始化为零），然后反复调整它们，沿着最陡下降的方向进行更新。

下图都展示了最陡下降方向逐步更新。





如果 $\frac{\partial \mathcal{J}}{\partial w_j} > 0 \implies$ 增加 w_j 会增加 $\mathcal{J}$,

$\frac{\partial \mathcal{J}}{\partial w_j} < 0 \implies$ 增加 w_j 会减少 $\mathcal{J}$.

梯度下降的更新规则为: $w_j \leftarrow w_j - \alpha \frac{\partial \mathcal{J}}{\partial w_j}$

对于足够小的学习率 α (除非 $\frac{\partial \mathcal{J}}{\partial w_j} = 0$), 这个更新规则总是减少成本函数。

$\alpha > 0$ 是学习率 (或步长)。学习率越大, 权重 w 的变化越快。

学习率的典型值很小, 例如 0.01 或 0.0001。

如果成本是 N 个个体损失的总和而不是它们的平均值, 则需要较小的学习率 ($\alpha' = \alpha/N$)。

梯度是成本函数 $\mathcal{J}$ 关于权重向量 $\mathbf{w}$ 的偏导数向量, 表示为 $\nabla_{\mathbf{w}} \mathcal{J} = \frac{\partial \mathcal{J}}{\partial \mathbf{w}} = \begin{pmatrix} \frac{\partial \mathcal{J}}{\partial w_1} \\ \vdots \\ \frac{\partial \mathcal{J}}{\partial w_D} \end{pmatrix}$ 这里梯度向量的

各个分量是成本函数关于每个权重的偏导数。梯度指向成本函数增加最快的方向。

梯度下降的更新规则可以表示为向量形式:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial \mathcal{J}}{\partial \mathbf{w}}$$

对于线性回归问题, 更新规则可以进一步展开为:

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\alpha}{N} \sum_{i=1}^N (y^{(i)} - t^{(i)}) \mathbf{x}^{(i)}$$

其中, N 是样本数量, $y^{(i)}$ 是第 i 个样本的真实标签, $t^{(i)}$ 是第 i 个样本的预测值, $\mathbf{x}^{(i)}$ 是第 i 个样本的特征向量。

梯度下降通过沿着梯度的反方向 (即成本函数减少最快的方向) 更新权重向量 $\mathbf{w}$

当梯度下降算法收敛时, 会达到一个临界点, 即梯度为零: $\frac{\partial \mathcal{J}}{\partial \mathbf{w}} = \mathbf{0}$

由于线性回归的平方误差损失是凸函数, 所以平方误差损失函数有一个全局最小值, 并且任何局部最小值也是全局最小值, 这使得优化问题更容易解决。

即使在线性回归中存在直接解法, 有时也需要使用梯度下降。那为什么这个时候需要使用梯度下降呢?

1. 梯度下降可以应用于更广泛的模型: 许多机器学习模型没有直接解法, 而梯度下降可以用于这些模型的优化。
2. 梯度下降可能比直接解法更容易实现: 对于某些复杂模型, 直接解法可能难以实现或计算成本高昂。

3. 在高维空间中，梯度下降比直接解法更有效：在特征数量 D 很大的情况下，直接解法可能非常耗时。

直接解法的公式为 $(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{t}$ 。

这里矩阵求逆是一个 $O(D^3)$ 算法，其中 D 是特征的数量。

每次梯度下降更新的成本是 $O(ND)$ ，其中 N 是样本数量， D 是特征数量。

当特征数量 D 远大于1时，直接解法的计算成本会显著高于梯度下降，特别是在高维空间中。

随机梯度下降（SGD）是一种梯度下降的变体，它在每次更新时只使用一个样本，可以进一步降低计算成本，这样每次更新的计算成本可能会更低。

梯度下降的更新规则可以表示为向量形式：

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial \mathcal{J}}{\partial \mathbf{w}}$$

当考虑L2正则化时，成本函数变为 $\mathcal{J} + \lambda \mathcal{R}$ ，其中 $\mathcal{R}$ 是L2正则化项， λ 是正则化参数。

梯度下降更新规则变为： $\mathbf{w} \leftarrow \mathbf{w} - \alpha \left(\frac{\partial \mathcal{J}}{\partial \mathbf{w}} + \lambda \frac{\partial \mathcal{R}}{\partial \mathbf{w}} \right)$

由于 $\mathcal{R}$ 是权重的平方和，其梯度是 $2\lambda \mathbf{w}$ ，因此更新规则进一步展开为： $\mathbf{w} \leftarrow \mathbf{w} - \alpha \left(\frac{\partial \mathcal{J}}{\partial \mathbf{w}} + \lambda \frac{\partial \mathcal{R}}{\partial \mathbf{w}} \right)$

$$= \mathbf{w} - \alpha \left(\frac{\partial \mathcal{J}}{\partial \mathbf{w}} + \lambda \mathbf{w} \right)$$

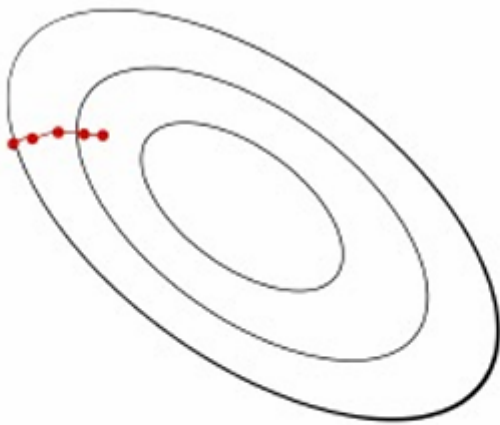
$$= (1 - \alpha\lambda) \mathbf{w} - \alpha \frac{\partial \mathcal{J}}{\partial \mathbf{w}}$$

更新规则中的 $(1 - \alpha\lambda) \mathbf{w}$ 部分表示权重衰减（weight decay）。这意味着每个权重在每次更新时都会稍微减少，这有助于防止权重变得过大，从而减少过拟合的风险。

1.5.3.1.1 学习率 α

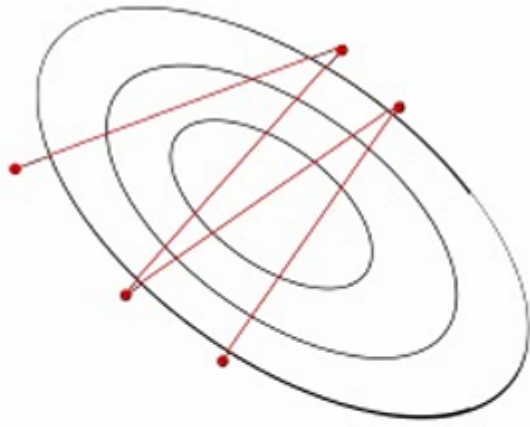
学习率 α 是梯度下降算法中的一个关键超参数，它决定了每次更新步长的大小。

当 α 过小时，算法的进展会变得非常缓慢。



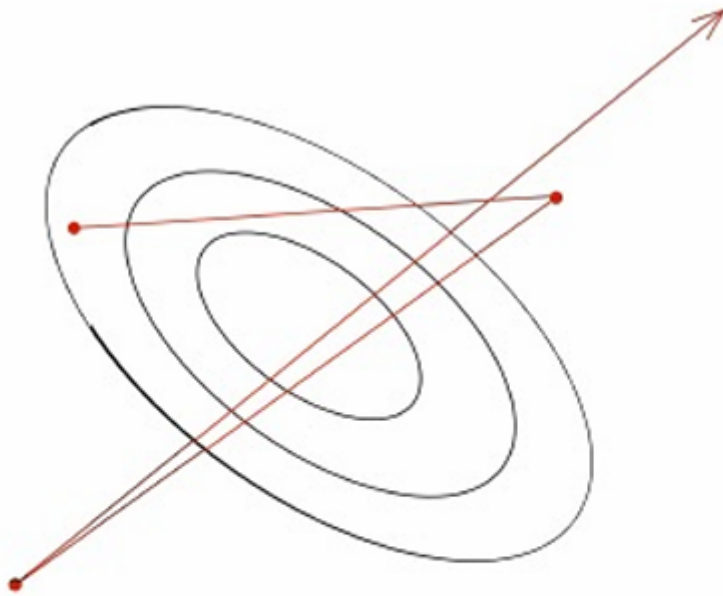
α too small:
slow progress

当 α 过大时，算法可能会在最优解附近来回振荡，无法稳定收敛。



α too large:
oscillations

当 α 过大时，算法可能会完全偏离最优解，甚至发散。



α much too large:
instability

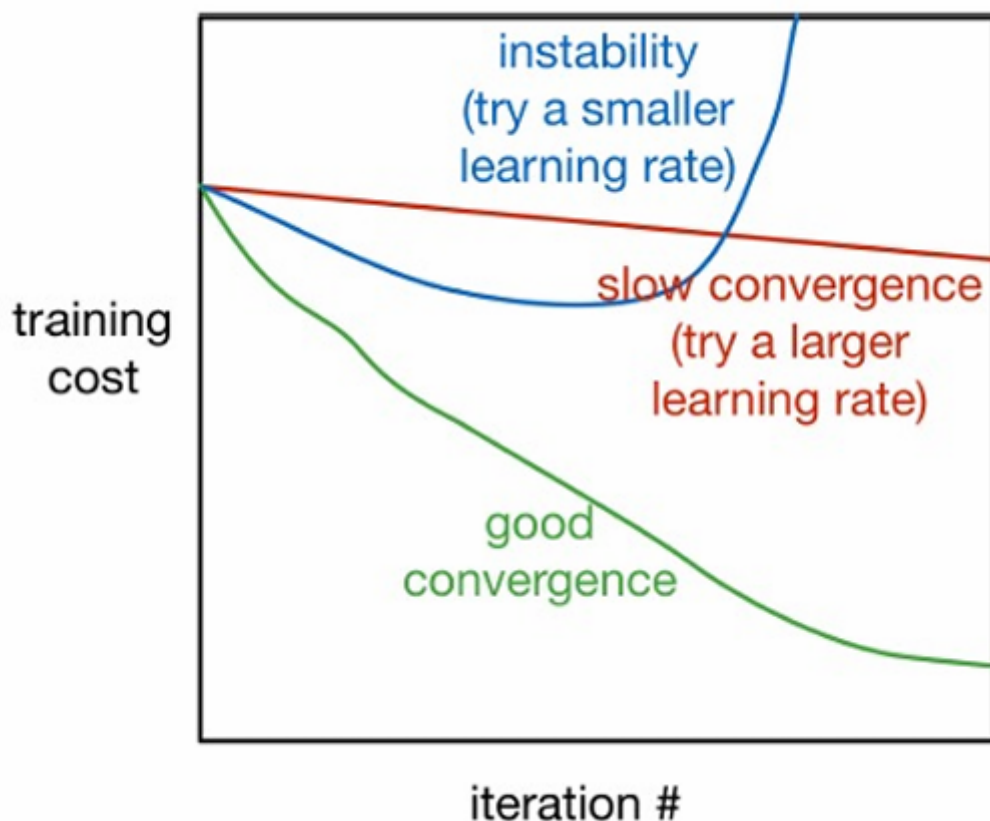
好的学习率值通常在0.001到0.1之间。

如果你希望获得良好的性能，可以通过网格搜索 (grid search)来尝试不同的学习率值，例如0.1、0.03、0.01等。

1.5.3.1.2 训练曲线 (Training Curves)

训练曲线是将训练成本（或损失）作为迭代次数（或训练轮数）的函数进行绘图，其可以帮助我们诊断和理解模型在训练过程中的表现。

下图展示了一个训练曲线。



不稳定性（蓝色曲线）：当学习率过大时，训练成本可能会在迭代过程中出现剧烈波动，导致模型不稳定，难以收敛到最优解。

收敛缓慢（红色曲线）：当学习率过小时，训练成本的下降速度会非常缓慢，导致模型收敛速度慢，需要更多的迭代次数才能达到较低的训练成本。

良好的收敛（绿色曲线）：当学习率设置得当时，训练成本会平稳且快速地下降，模型能够较快地收敛到较低的训练成本。

需要注意训练曲线可以说明学习率设置不当导致的不稳定性或收敛缓慢，但它们并不能保证模型已经收敛到全局最优解。

1.5.3.1.3 随机梯度下降（Stochastic Gradient Descent, SGD）

前面我们提到了随机梯度下降可以降低计算成本。

成本函数 $\mathcal{J}$ 是训练样本上平均损失的函数：

$$\mathcal{J}(\theta) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}^{(i)} = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y(x^{(i)}, \theta), t^{(i)})$$

这里， θ 表示模型参数（例如，在线性回归中， $\theta = (\mathbf{w}, b)$ ）， $\mathcal{L}$ 是损失函数， $y(x^{(i)}, \theta)$ 是模型对第 i 个样本的预测值， $t^{(i)}$ 是第 i 个样本的真实标签。

根据线性性质，成本函数关于参数 θ 的梯度可以表示为：

$$\frac{\partial \mathcal{J}}{\partial \theta} = \frac{1}{N} \sum_{i=1}^N \frac{\partial \mathcal{L}^{(i)}}{\partial \theta}$$

计算梯度需要对所有训练样本求和，这种方法称为批量训练。当数据集非常大（例如，包含数百万训练样本）时，批量训练可能变得不切实际，因为计算和存储整个数据集的梯度会非常耗时和占用大量内存。

随机梯度下降是一种优化算法，它在每次更新时只使用一个训练样本（或一个小批量样本）来计算梯度。这种方法可以显著减少每次更新的计算成本，使得算法能够更快地进行更新，并且更有效地处理大规模数据集。

更新规则分为两步：

1. 从训练数据中随机均匀选择一个样本索引 i 。
2. 使用该样本的梯度来更新参数 θ : $\theta \leftarrow \theta - \alpha \frac{\partial \mathcal{L}^{(i)}}{\partial \theta}$
这里, α 是学习率, $\frac{\partial \mathcal{L}^{(i)}}{\partial \theta}$ 是第 i 个样本的损失函数关于参数 θ 的梯度。

每次SGD更新的成本与样本数量 N 无关。这意味着即使数据集很大, 每次更新的计算成本也保持不变。这使得SGD在处理大规模数据集时更加高效。

如果随机均匀地采样一个训练样本, 那么随机梯度是批量梯度的无偏估计:

$$\mathbb{E} \left[\frac{\partial \mathcal{L}^{(i)}}{\partial \theta} \right] = \frac{1}{N} \sum_{i=1}^N \frac{\partial \mathcal{L}^{(i)}}{\partial \theta} = \frac{\partial \mathcal{J}}{\partial \theta}$$

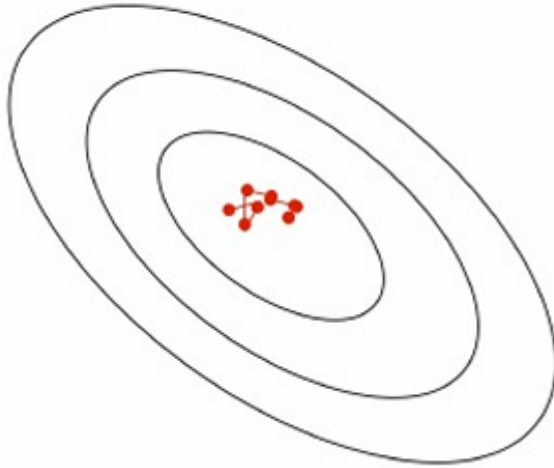
这里, $\mathbb{E}$ 表示期望值, $\frac{\partial \mathcal{J}}{\partial \theta}$ 是整个数据集的梯度。

1.5.3.1.3.1 学习率对SGD的影响

在随机梯度下降中, 学习率不仅影响算法的收敛速度, 还影响由于梯度的随机性导致的算法的波动 (fluctuations)。

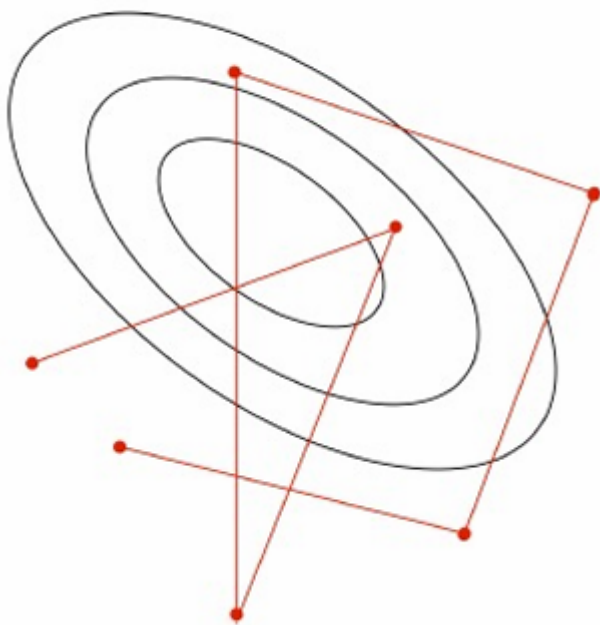
当学习率较小时, 每次更新的步长较小, 这会导致算法在接近最优解时波动较小, 但收敛速度慢。

Small learning rate



当学习率较大时, 每次更新的步长较大, 这可能导致算法在最优解附近产生较大的波动, 甚至可能无法稳定收敛。

Large learning rate



因此我们可以在训练的早期阶段使用较大

的学习率，以便快速接近最优解，然后随着训练的进行，逐渐减小学习率，以减少算法在最优解附近的波动，帮助算法更稳定地收敛到最优解。

1.5.3.1.4 小批量 (mini-batch) 梯度下降

使用单个训练样本计算的梯度估计可能具有高方差，这意味着估计值可能会有较大的波动。而且单个样本的梯度计算无法充分利用现代硬件（如GPU）的向量化计算能力，这可能导致计算效率低下。

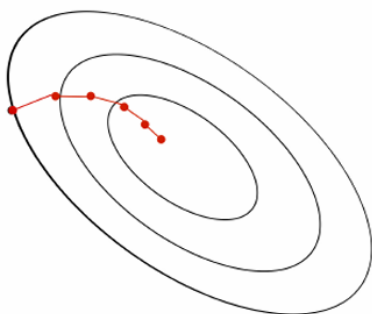
小批量梯度下降在计算梯度时不是使用单个样本，而是随机选择一个中等大小的训练样本集合，这个集合称为小批量 $\mathcal{M} \subset 1, \dots, N$ 。

这样在较大的小批量上计算的随机梯度具有较小的方差。

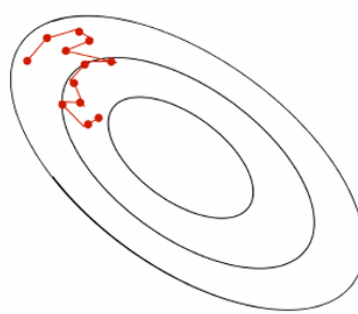
这里 $\mathcal{M}$ 是一个需要设置的超参数，同样需要仔细选择。

如果太大，那就需要更多的计算资源。如果太小就无法充分利用向量化操作，梯度估计的方差仍然较高。一个合理的小批量大小可能是 $|\mathcal{M}| = 100$ 。

下图展示了批量梯度下降（Batch Gradient Descent）和随机梯度下降（Stochastic Gradient Descent, SGD）两种优化算法。



batch gradient descent



stochastic gradient descent

在批量梯度下降中，每一步更新都是基于整个训练集计算得到的梯度。这意味着每一步都是朝着成本函数局部最小值最陡峭的方向移动。

因此，批量梯度下降的路径通常比较平滑，直接向山下（即成本函数的最小值）移动。

在SGD中，每一步更新只基于一个随机选择的训练样本计算得到的梯度。由于只使用了单个样本，计算得到的梯度估计会有较大的方差，导致更新方向可能会有较大的随机性。尽管每一步的更新方向可能比较“嘈杂”（即随机性较大），但从长远来看，SGD仍然会朝着成本函数最小值的方向移动，只是路径可能会比较曲折。